

Path Compression

Path compression is an optimization technique used in [[Union Find|union-find]] data structures. When finding the representative of an element, compress the path by redirecting every node along the path to point directly to the root.

Natural Explanation

In naive union-find, parent pointers can form a deep chain: $n_1 \rightarrow n_2 \rightarrow n_3 \rightarrow \dots \rightarrow \text{root}$. Each FIND operation traverses this chain, taking time proportional to chain length.

Path compression flattens the chain: during a FIND, make every node on the path point directly to the root. Future FINDs are much faster.

Standard Implementation

Modify the FIND operation to compress:

```
FIND(i)
    if i != p[i]:
        p[i] = FIND(p[i]) // Recursively find root and compress
    return p[i]
```

When recursion returns, $p[i]$ is set to the root (returned by recursive call), not the original parent.

Effect: every node visited during the search now points directly to the root.

Iterative Path Compression

```
FIND(i)
    root = i
    while root != p[root]:
        root = p[root]

    // Compress by redirecting all nodes on path
    while i != root:
        next = p[i]
        p[i] = root
        i = next

    return root
```

Impact on Union-Find Complexity

With path compression alone (no union optimization): - Worst-case single operation: $O(\log n)$ - Amortized per operation: $O(\log n)$

With path compression and union by rank/size: - Amortized per operation: $O(\alpha(n))$ where α is the inverse Ackermann function - In practice, $\alpha(n) \leq 4$ for all practical n , so operations are essentially $O(1)$

Without path compression (e.g., naive Quick Union): - Single operation: $O(n)$ worst case - Makes a huge difference

Why It Works

Path compression maintains the union-find invariant (each component has a unique root) while flattening trees: - A root still points to itself - Non-root nodes get a shorter path to their root - The component structure is unchanged

Key property: if you compress a tree, the tree gets flatter but remains a valid union-find tree.

Intuition

Early operations may be slow (building up the tree), but later operations compress paths. Over many operations, most paths are short, so amortized cost is very low.

This is an example of **amortized analysis**: total cost spread over many operations is much better than worst case.

Related

- [[Union Find]]
- [[Amortized Analysis]]